There is, however, also something that might look surprising: `_frequency_table_to_heap` seems to improve with larger branching factors, while `_heap_to_tree` has a minimum for branching factors around 9 and 10.

As explained in section 2.6, which shows the pseudocode implementation of heap methods (and as you can see from the code on GitHub), the former method only calls the`_push_down` helper method, while the latter uses both `top` (which in turn calls `_push_down`) and `insert` (relying on `_bubble_up` instead), so we would expect to see the opposite result. It is anyway true that even `_heap_to_tree` has a ratio of 2 calls to `top` per `insert`.

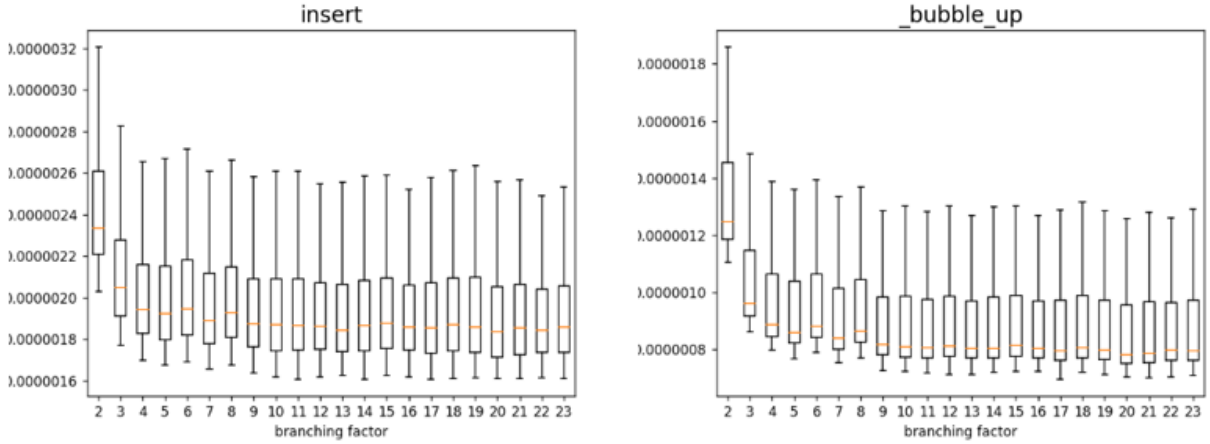


Figure 2.21 The distribution of recorded running times per-call for `insert` and `_bubble_up`.

Let’s then delve into the internals of these high-level methods, and see the running time per-call of heap’s internal method `_heapify` (figure 2.22), of API’s methods `top` and `insert` and their helper methods `_push_down` and `_bubble_up` (figure 2.21 and 2.23).